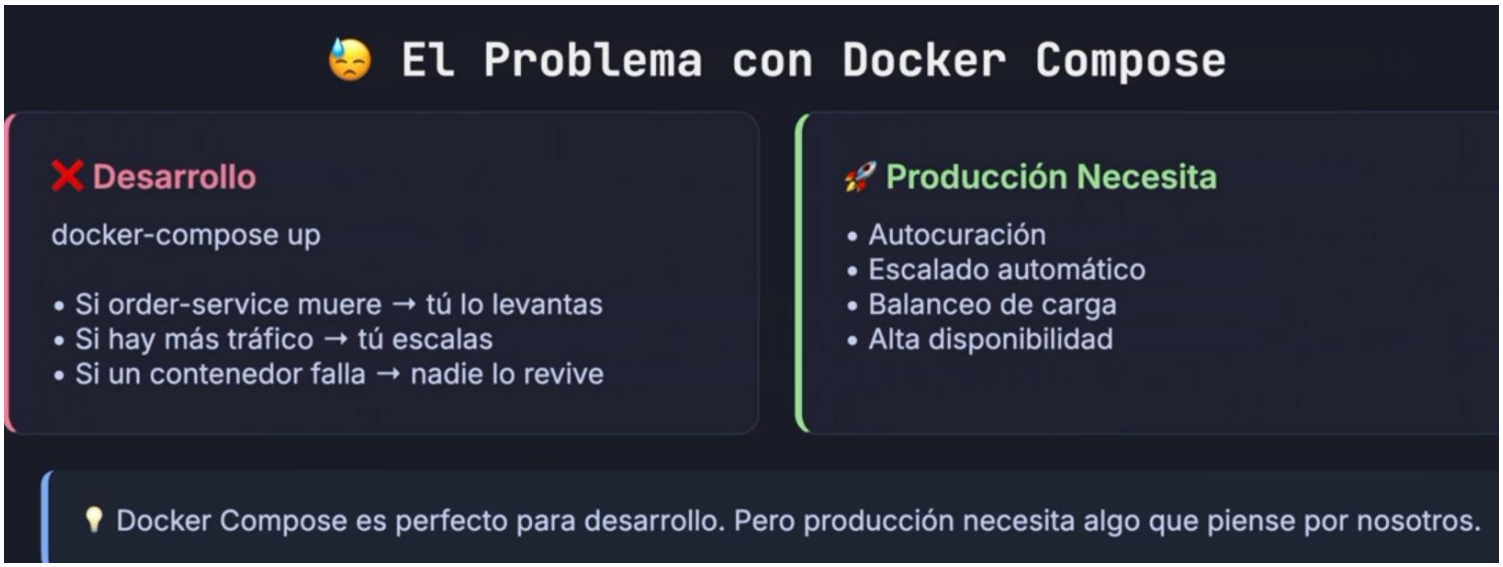
Sección 12: De Docker Compose a Kubernetes: Deploy Completo en Producción

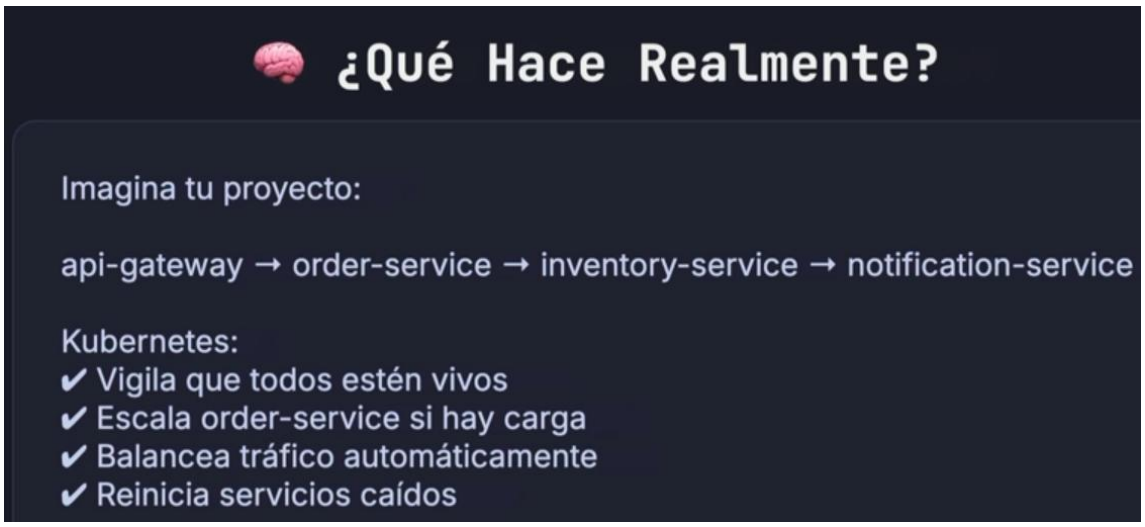
- **El cambio conceptual:** Reemplazo de Eureka por el DNS nativo de Kubernetes y del Config Server por variables de entorno y perfiles application-k8s.yml.
- **Dockerfiles Multistage:** Optimización de imágenes de producción con builds en dos etapas para reducir el tamaño final.
- **Recursos de Kubernetes:** Comprensión práctica de Deployments, Services y PersistentVolumeClaims como bloques fundamentales del cluster.
- **Infraestructura en K8s:** Despliegue de bases de datos, RabbitMQ y Keycloak con sus volúmenes persistentes y configuración de DNS interno.
- **Inyección de imágenes en Minikube:** Ciclo completo de construcción e inyección de imágenes locales en el cluster.
- **Keycloak en Kubernetes:** Export del realm, ConfigMap para importación automática y resolución del problema de hostname en Mac.
- **Observabilidad en K8s:** Activación de OpenTelemetry mediante variables de entorno para conectar los microservicios con Grafana LGTM dentro del cluster.
- **Prueba end-to-end:** Verificación del sistema completo desde Postman a través del gateway hasta RabbitMQ y el notification-service.

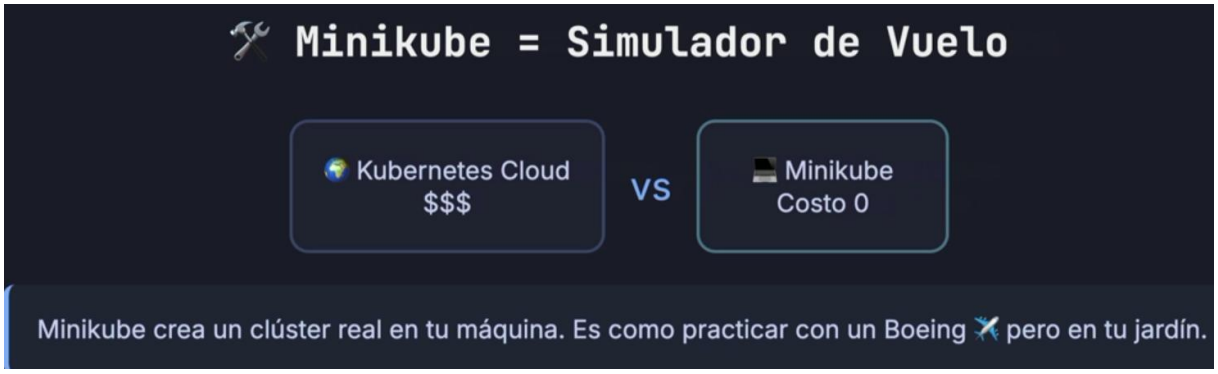
➔Kubernetes tiene su propio DNS, por ello se puede prescindir de Eureka. Tiene sus propias configuraciones (environments), se puede prescindir de config-server.

168. Kubernetes: El director de orquesta




Kubernetes = Director de Orquesta
Tú
"Quiero 3 order-service"
Kubernetes
Estado Deseado
3 Pods corriendo
Si un Pod muere... Kubernetes lo vuelve a crear automáticamente


¿Qué Hace Realmente?
Imagina tu proyecto:
api-gateway → order-service → inventory-service → notification-service
Kubernetes:
✔ Vigila que todos estén vivos
✔ Escala order-service si hay carga
✔ Balancea tráfico automáticamente
✔ Reinicia servicios caídos


Minikube = Simulador de Vuelo
Kubernetes Cloud
$$$
VS
Minikube
Costo 0
Minikube crea un clúster real en tu máquina. Es como practicar con un Boeing ✈️ pero en tu jardín.


Conceptos que Sí Necesitas
Pod
Contenedor Spring Boot
Nodo
Máquina que ejecuta Pods
Deployment
Cantidad de réplicas
Service
Expone el microservicio
No vamos a aprender TODO Kubernetes. Solo lo necesario para llevar tu arquitectura a producción.

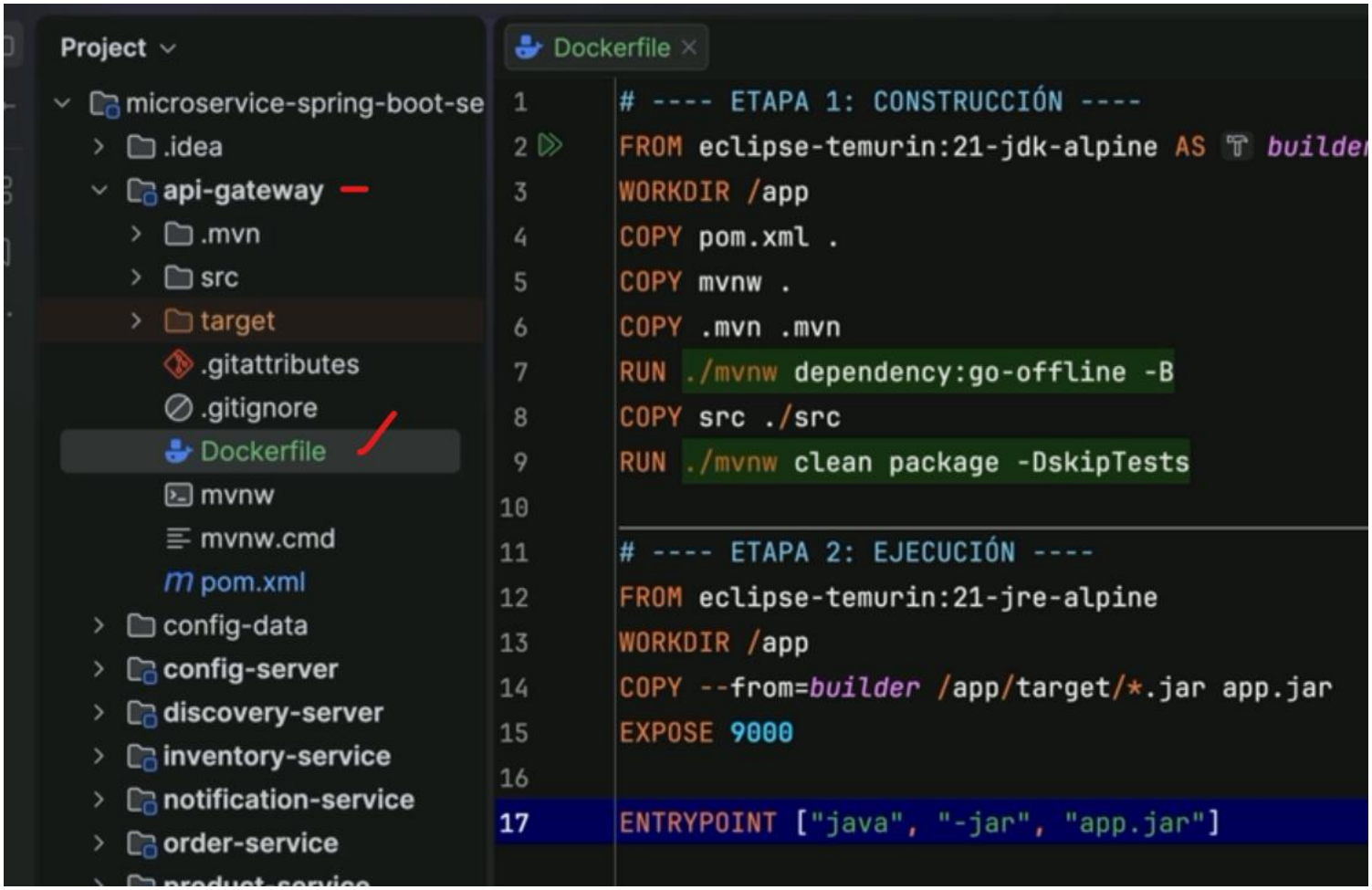
169. Instalación de laboratorio

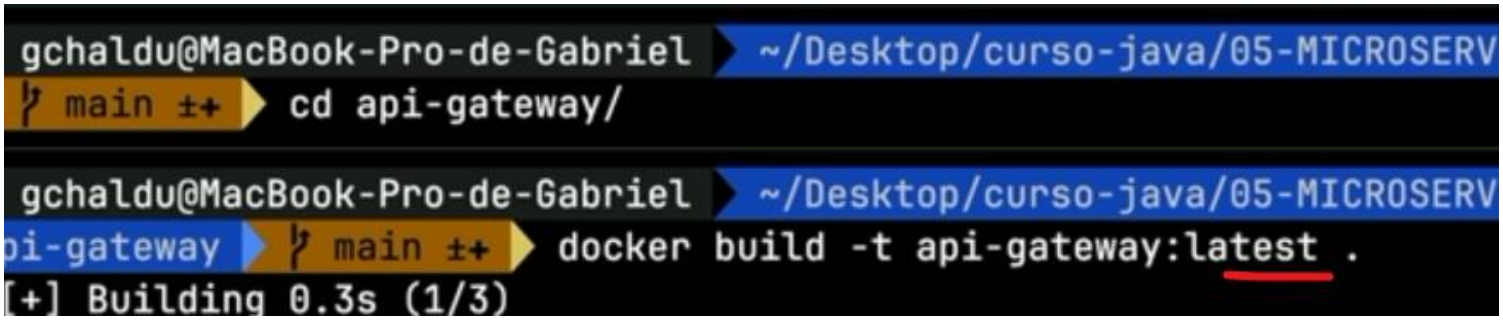
https://gist.github.com/gchaldu/55427fe1dd4667553e651fa7d5cf3235

https://minikube.sigs.k8s.io/docs/

https://kubernetes.io/es/docs/tutorials/hello-minikube/

170. El Dockerfile: Multistage





➔Repetir el Dockerfile en inventory-service (cambiando el puerto al 8083), order-service (8082), product-service (8081), notification-service (8084). Realizar u docker build...

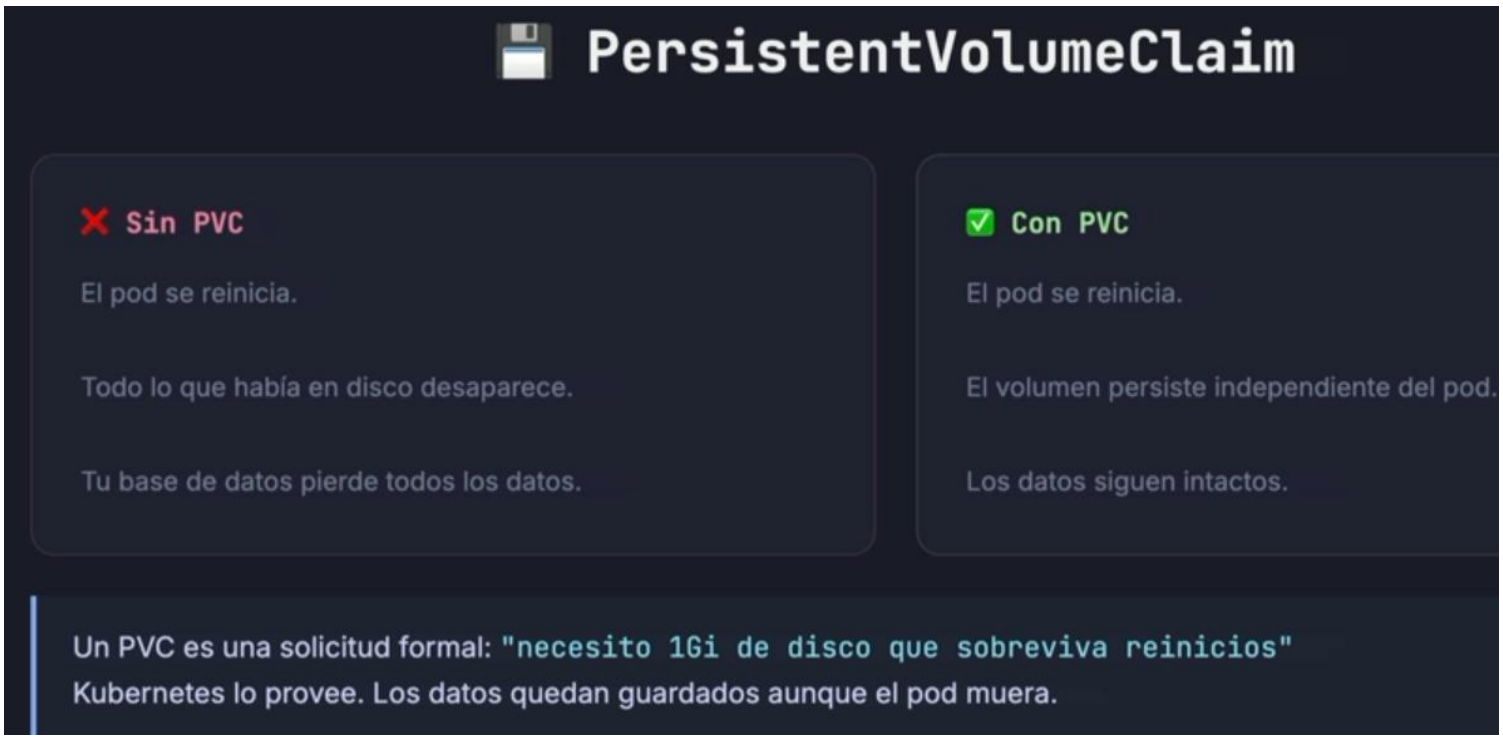
```
gchaldu@MacBook-Pro-de-Gabriel ~/Desktop/curso-java/05-MICROSERVICIOS/PROYECTOS/12-prueba/microservice-spring-boot-section-8-main
 main ±+ docker images | grep -E "api-gateway|inventory|order-service|product|notification"
notification-service                  latest                                    d47055e5d0f6
 31 seconds ago       384MB
product-service                       latest                                    0198ba0c930a
 About a minute ago   393MB
order-service                         latest                                    647903151727
 2 minutes ago        513MB
inventory-service                     latest                                    8e6d42ea0688
 4 minutes ago        472MB
api-gateway                           latest                                    40ae940023a8
 19 minutes ago       436MB
```

172. Conceptos base y PVCs

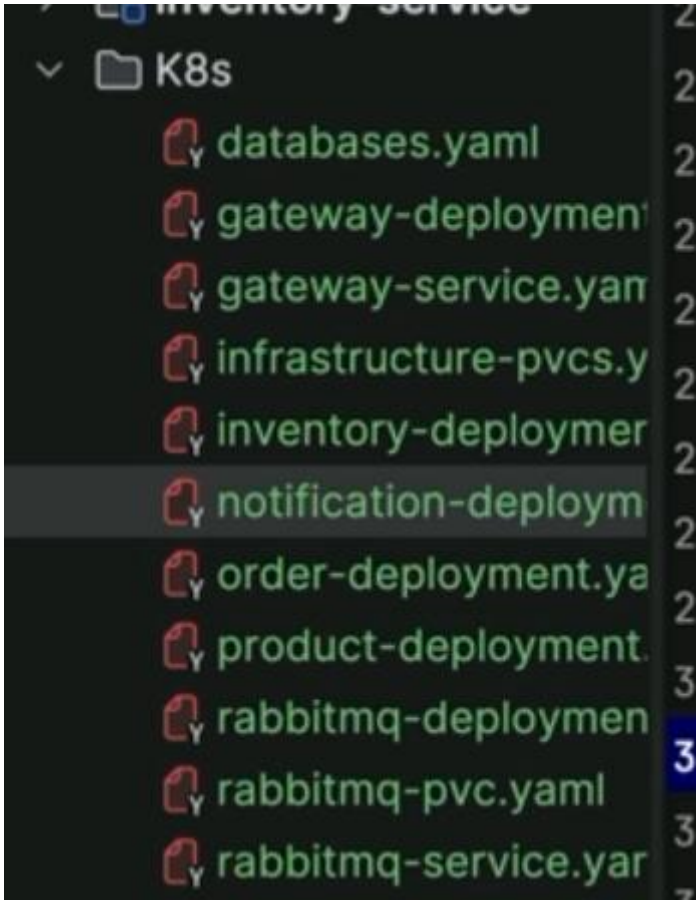
https://gist.github.com/gchaldu/67b45a2d87eee6e1b116181fb447032e

https://gist.github.com/gchaldu/ff4bd796f0d67f5cef136c437e7f0c5c

➔Todos los archivos de configuración de kubernetes, estarán en la carpeta k8s en la raíz del proyecto.



➔En la carpeta k8s:



infrastructure-pvcs.yaml ➔Almacenamiento, se reserva un espacio en el disco

```
apiVersion: v1
kind: List
items:
 - apiVersion: v1
   kind: PersistentVolumeClaim
   metadata:
    name: mongo-pvc
   spec:
    accessModes: [ReadWriteOnce]
    resources: { requests: { storage: 1Gi } }
 - apiVersion: v1
   kind: PersistentVolumeClaim
   metadata:
    name: inventory-db-pvc
   spec:
    accessModes: [ReadWriteOnce]
    resources: { requests: { storage: 1Gi } }
 - apiVersion: v1
```

```
  kind: PersistentVolumeClaim
  metadata:
   name: order-db-pvc
  spec:
   accessModes: [ReadWriteOnce]
   resources: { requests: { storage: 1Gi } }
- apiVersion: v1
  kind: PersistentVolumeClaim
  metadata:
   name: keycloak-db-pvc
  spec:
   accessModes: [ReadWriteOnce]
   resources: { requests: { storage: 1Gi } }
```

➔rabbitmq-pvc.yaml ➔Separado, ya que no es BD sino una infraestructura de mensajería

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
 name: rabbitmq-pvc
spec:
 accessModes:
  - ReadWriteOnce
 resources:
  requests:
   storage: 1Gi
```

173. Bases de datos y RabbitMQ

➔databases.yaml

```
# 1. MONGODB (Product Service)
apiVersion: apps/v1
kind: Deployment
metadata:
```

```
  name: mongodb
spec:
  selector:
    matchLabels:
      app: mongodb
  template:
    metadata:
      labels:
        app: mongodb
    spec:
      containers:
        - name: mongodb
          image: mongo:7.0.4
          ports:
            - containerPort: 27017
          env:
            - name: MONGO_INITDB_ROOT_USERNAME
              value: "root"
            - name: MONGO_INITDB_ROOT_PASSWORD
              value: "password"
          volumeMounts:
            - name: mongo-storage
              mountPath: /data/db
      volumes:
        - name: mongo-storage
          persistentVolumeClaim:
            claimName: mongo-pvc
---
apiVersion: v1
kind: Service
metadata:
  name: mongodb
```

```
spec:
  selector:
    app: mongodb
  ports:
    - port: 27017
      targetPort: 27017
---
# 2. MYSQL (Inventory Service)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: inventory-db
spec:
  selector:
    matchLabels:
      app: inventory-db
  template:
    metadata:
      labels:
        app: inventory-db
    spec:
      containers:
        - name: mysql
          image: mysql:8
          ports:
            - containerPort: 3306
          env:
            - name: MYSQL_ROOT_PASSWORD
              value: "root"
            - name: MYSQL_DATABASE
              value: "inventory-db"
          volumeMounts:
```

```
        - name: mysql-storage
          mountPath: /var/lib/mysql
      volumes:
       - name: mysql-storage
         persistentVolumeClaim:
          claimName: inventory-db-pvc
---
apiVersion: v1
kind: Service
metadata:
  name: inventory-db
spec:
  selector:
    app: inventory-db
  ports:
    - port: 3306
      targetPort: 3306
---
# 3. POSTGRES (Order Service)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-db
spec:
  selector:
    matchLabels:
      app: order-db
  template:
    metadata:
      labels:
        app: order-db
    spec:
```

```
    containers:
      - name: postgres
        image: postgres:16-alpine
        ports:
          - containerPort: 5432
        env:
          - name: POSTGRES_USER
            value: "admin"
          - name: POSTGRES_PASSWORD
            value: "admin"
          - name: POSTGRES_DB
            value: "order-db"
        volumeMounts:
          - name: order-storage
            mountPath: /var/lib/postgresql/data
    volumes:
      - name: order-storage
        persistentVolumeClaim:
          claimName: order-db-pvc
---
apiVersion: v1
kind: Service
metadata:
  name: order-db
spec:
  selector:
    app: order-db
  ports:
    - port: 5432
      targetPort: 5432
---
# 4. POSTGRES (Keycloak DB)
```

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
 name: keycloak-db
spec:
 selector:
  matchLabels:
   app: keycloak-db
 template:
  metadata:
   labels:
    app: keycloak-db
  spec:
   containers:
    - name: postgres
      image: postgres:16-alpine
      ports:
       - containerPort: 5432
      env:
       - name: POSTGRES_USER
         value: "keycloak"
       - name: POSTGRES_PASSWORD
         value: "password"
       - name: POSTGRES_DB
         value: "keycloak-db"
      volumeMounts:
       - name: keycloak-db-storage
         mountPath: /var/lib/postgresql/data
   volumes:
    - name: keycloak-db-storage
      persistentVolumeClaim:
       claimName: keycloak-db-pvc
```

```
---
apiVersion: v1
kind: Service
metadata:
  name: keycloak-db
spec:
  selector:
    app: keycloak-db
  ports:
    - port: 5432
      targetPort: 5432
```

➔rabbitmq-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: rabbitmq-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: rabbitmq
  template:
    metadata:
      labels:
        app: rabbitmq
    spec:
      containers:
        - name: rabbitmq
          image: rabbitmq:4.2-management
          ports:
            - containerPort: 5672
```

```
        - containerPort: 15672
        env:
        - name: RABBITMQ_DEFAULT_USER
          value: "guest"
        - name: RABBITMQ_DEFAULT_PASS
          value: "guest"
        volumeMounts:
        - name: rabbitmq-data
          mountPath: /var/lib/rabbitmq
    volumes:
     - name: rabbitmq-data
       persistentVolumeClaim:
        claimName: rabbitmq-pvc
```

➔rabbitmq-service.yaml

```
apiVersion: v1
kind: Service
metadata:
 name: rabbitmq
spec:
 type: NodePort
 selector:
  app: rabbitmq
 ports:
  - name: amqp
    port: 5672
    targetPort: 5672
  - name: http
    port: 15672
    targetPort: 15672
    nodePort: 31672
```

174. Inyectando imágenes y levantando la infraestructura

```
gchaldu@MacBook-Pro-de-Gabriel ~/Desktop/curso-java/05
main ±+ minikube image load api-gateway:latest
```

```
gchaldu@MacBook-Pro-de-Gabriel ~/Desktop/curso-java/05-MI
main ±+ minikube image load product-service:latest
```

```
gchaldu@MacBook-Pro-de-Gabriel ~/Desktop/curso-java/05-M
main ±+ minikube image load order-service:latest
```

```
gchaldu@MacBook-Pro-de-Gabriel ~/Desktop/curso-java/05-MICROSE
main ±+ minikube image load inventory-service:latest
gchaldu@MacBook-Pro-de-Gabriel ~/Desktop/curso-java/05-MICROSE
main ±+ minikube image load notification-service:latest
```

```
gchaldu@MacBook-Pro-de-Gabriel ~/Desktop
main ±+ kubectl apply -f K8s/
```

```
gchaldu@MacBook-Pro-de-Gabriel ~/Desktop/curso-java/05-MICROSERVICIOS/PRO
main ±+ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
inventory-db-7c79cd64dc-8t8w7       1/1     Running   0          2m3s
keycloak-db-574f6f6bd-6rt97         1/1     Running   0          2m3s
mongodb-5db5898c5b-ptdr4            1/1     Running   0          2m3s
order-db-fcbcd66c5-smwnc            1/1     Running   0          2m3s
rabbitmq-deployment-769b94f458-6jvkh 1/1    Running   0          2m3s
```

175. Los deployments de los Microservicios 1

➔gateway-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: gateway-deployment
  labels:
    app: api-gateway
spec:
```

```
replicas: 1
selector:
 matchLabels:
  app: api-gateway
template:
 metadata:
  labels:
   app: api-gateway
 spec:
  containers:
   - name: api-gateway
     image: api-gateway:latest
     imagePullPolicy: Never
     ports:
      - containerPort: 9000
     env:
      - name: SPRING_PROFILES_ACTIVE
        value: "k8s"
      - name: SERVER_PORT
        value: "9000"
      - name: SPRING_CLOUD_CONFIG_ENABLED
        value: "false"
      - name: EUREKA_CLIENT_ENABLED
        value: "false"
      - name: SPRING_CLOUD_DISCOVERY_ENABLED
        value: "false"
      - name: MANAGEMENT_OPENTELEMETRY_TRACING_EXPORT_OTLP_ENDPOINT
        value: "http://lgtm:4318/v1/traces"
      - name: MANAGEMENT_OPENTELEMETRY_LOGGING_EXPORT_OTLP_ENDPOINT
        value: "http://lgtm:4318/v1/logs"
      - name: MANAGEMENT_OTLP_METRICS_EXPORT_URL
        value: "http://lgtm:4318/v1/metrics"
```

```
        - name: MANAGEMENT_TRACING_SAMPLING_PROBABILITY
          value: "1.0"
        - name: SPRING_SECURITY_OAUTH2_RESOURCESERVER_JWT_ISSUER_URI
          value: "http://keycloak:8080/realms/ecommerce-realm"
```

➔gateway-service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: gateway-service
spec:
  type: NodePort
  selector:
    app: api-gateway
  ports:
    - port: 9000
      targetPort: 9000
      nodePort: 30000
```

➔product-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: product-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: product-service
  template:
    metadata:
      labels:
```

```
    app: product-service
  spec:
    containers:
      - name: product-service
        image: product-service:latest
        imagePullPolicy: Never
        ports:
          - containerPort: 8081
        env:
          - name: SPRING_PROFILES_ACTIVE
            value: "k8s"
          - name: SERVER_PORT
            value: "8081"
          - name: SPRING_CLOUD_CONFIG_ENABLED
            value: "false"
          - name: EUREKA_CLIENT_ENABLED
            value: "false"
          - name: SPRING_SECURITY_OAUTH2_RESOURCESERVER_JWT_ISSUER_URI
            value: "http://keycloak:8080/realms/ecommerce-realm"
          - name: SPRING_DATA_MONGODB_HOST
            value: "mongodb"
          - name: SPRING_DATA_MONGODB_PORT
            value: "27017"
          - name: SPRING_DATA_MONGODB_DATABASE
            value: "product-db"
          - name: SPRING_DATA_MONGODB_USERNAME
            value: "root"
          - name: SPRING_DATA_MONGODB_PASSWORD
            value: "password"
          - name: SPRING_DATA_MONGODB_AUTHENTICATION_DATABASE
            value: "admin"
---
```

```
apiVersion: v1
kind: Service
metadata:
  name: product-service
spec:
  selector:
    app: product-service
  ports:
    - port: 8081
      targetPort: 8081
```

➔order-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: order-service
  template:
    metadata:
      labels:
        app: order-service
    spec:
      containers:
        - name: order-service
          image: order-service:latest
          imagePullPolicy: Never
          ports:
            - containerPort: 8082
```

```
        env:
          - name: SPRING_PROFILES_ACTIVE
            value: "k8s"
          - name: SERVER_PORT
            value: "8082"
          - name: SPRING_CLOUD_CONFIG_ENABLED
            value: "false"
          - name: EUREKA_CLIENT_ENABLED
            value: "false"
          - name: SPRING_SECURITY_OAUTH2_RESOURCESERVER_JWT_ISSUER_URI
            value: "http://keycloak:8080/realms/ecommerce-realm"
          - name: MANAGEMENT_OPENTELEMETRY_TRACING_EXPORT_OTLP_ENDPOINT
            value: "http://lgtm:4318/v1/traces"
          - name: MANAGEMENT_OPENTELEMETRY_LOGGING_EXPORT_OTLP_ENDPOINT
            value: "http://lgtm:4318/v1/logs"
          - name: MANAGEMENT_OTLP_METRICS_EXPORT_URL
            value: "http://lgtm:4318/v1/metrics"
          - name: MANAGEMENT_TRACING_SAMPLING_PROBABILITY
            value: "1.0"
          - name: SPRING_DATASOURCE_URL
            value: "jdbc:postgresql://order-db:5432/order-db"
          - name: SPRING_DATASOURCE_USERNAME
            value: "admin"
          - name: SPRING_DATASOURCE_PASSWORD
            value: "admin"
          - name: SPRING_RABBITMQ_HOST
            value: "rabbitmq"
          - name: SPRING_JPA_HIBERNATE_DDL_AUTO
            value: "update"
---
apiVersion: v1
kind: Service
```

```
metadata:
  name: order-service
spec:
  selector:
    app: order-service
  ports:
    - port: 8082
      targetPort: 8082
```

176. Los deployments de los Microservicios 2

➔inventory-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: inventory-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: inventory-service
  template:
    metadata:
      labels:
        app: inventory-service
    spec:
      containers:
        - name: inventory-service
          image: inventory-service:latest
          imagePullPolicy: Never
          ports:
            - containerPort: 8083
          env:
```

```
        - name: SPRING_PROFILES_ACTIVE
          value: "k8s"
        - name: SERVER_PORT
          value: "8083"
        - name: SPRING_CLOUD_CONFIG_ENABLED
          value: "false"
        - name: EUREKA_CLIENT_ENABLED
          value: "false"
        - name: SPRING_SECURITY_OAUTH2_RESOURCESERVER_JWT_ISSUER_URI
          value: "http://keycloak:8080/realms/ecommerce-realm"
        - name: MANAGEMENT_OPENTELEMETRY_TRACING_EXPORT_OTLP_ENDPOINT
          value: "http://lgtm:4318/v1/traces"
        - name: MANAGEMENT_OPENTELEMETRY_LOGGING_EXPORT_OTLP_ENDPOINT
          value: "http://lgtm:4318/v1/logs"
        - name: MANAGEMENT_OTLP_METRICS_EXPORT_URL
          value: "http://lgtm:4318/v1/metrics"
        - name: MANAGEMENT_TRACING_SAMPLING_PROBABILITY
          value: "1.0"
        - name: SPRING_DATASOURCE_URL
          value: "jdbc:mysql://inventory-db:3306/inventory-db"
        - name: SPRING_DATASOURCE_USERNAME
          value: "root"
        - name: SPRING_DATASOURCE_PASSWORD
          value: "root"
        - name: SPRING_RABBITMQ_HOST
          value: "rabbitmq"
        - name: SPRING_JPA_HIBERNATE_DDL_AUTO
          value: "update"
---
apiVersion: v1
kind: Service
metadata:
```

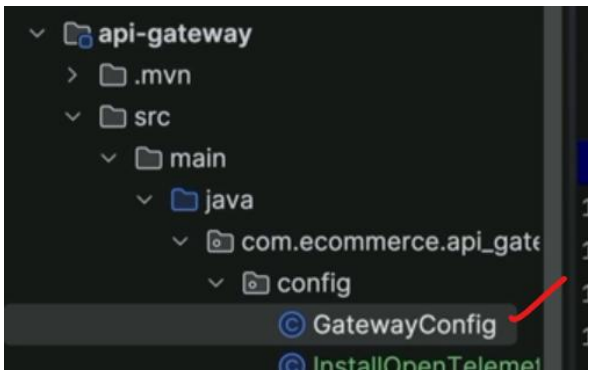
```
  name: inventory-service
spec:
  selector:
    app: inventory-service
  ports:
    - port: 8083
      targetPort: 8083
```

➔notification-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: notification-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: notification-service
  template:
    metadata:
      labels:
        app: notification-service
    spec:
      containers:
        - name: notification-service
          image: notification-service:latest
          imagePullPolicy: Never
          ports:
            - containerPort: 8084
          env:
            - name: SPRING_PROFILES_ACTIVE
              value: "k8s"
            - name: SERVER_PORT
```

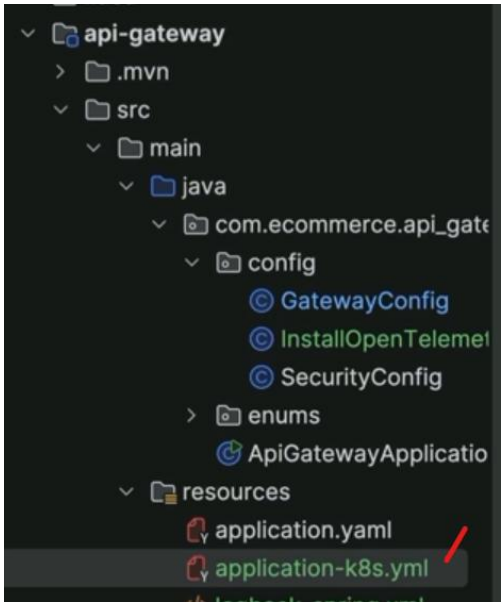
```
          value: "8084"
        - name: SPRING_CLOUD_CONFIG_ENABLED
          value: "false"
        - name: EUREKA_CLIENT_ENABLED
          value: "false"
        - name: SPRING_RABBITMQ_HOST
          value: "rabbitmq"
        - name: SPRING_MAIL_HOST
          value: "sandbox.smtp.mailtrap.io"
        - name: SPRING_MAIL_PORT
          value: "2525"
        - name: SPRING_MAIL_USERNAME
          value: "TU_USERNAME_MAILTRAP"
        - name: SPRING_MAIL_PASSWORD
          value: "TU_PASSWORD_MAILTRAP"
---
apiVersion: v1
kind: Service
metadata:
  name: notification-service
spec:
  selector:
    app: notification-service
  ports:
    - port: 8084
      targetPort: 8084
```

177. El gran cambio: De Eureka a Kubernetes




```java
@Configuration
public class GatewayConfig {

    @Bean
    public RouteLocator routeLocator(RouteLocatorBuilder builder){
        return builder.routes()
                .route("product-service", r -> r
                        .path("/api/v1/product/**")
                        .uri("http://product-service:8081"))
                .route("order-service", r -> r
                        .path("/api/v1/order/**")
                        .uri("http://order-service:8082"))
                .route("inventory-service", r -> r
                        .path("/api/v1/inventory/**")
                        .uri("http://inventory-service:8083"))
                .build();
    }
}
```



```yaml
spring:
  cloud:
    config:
      enabled: false
    discovery:
      enabled: false
    gateway:
      routes:
        - id: product-service
          uri: http://product-service:8081
          predicates:
            - Path=/api/v1/product/**
        - id: order-service
          uri: http://order-service:8082
          predicates:
            - Path=/api/v1/order/**
        - id: inventory-service
          uri: http://inventory-service:8083
          predicates:
            - Path=/api/v1/inventory/**
  rabbitmq:
    host: rabbitmq

security:
  oauth2:
    resourceserver:
      jwt:
        issuer-uri: http://keycloak:8080/realms/ecommerce-realm
```

➔Reconstruir la imagen: